AICTE VERIFIED STUDENT REGISTER

Vibecoding Specification Pack

How to attach the right PDFs, prompt the coding agent, and keep implementation honest.

Document: 00-READ-ME Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Purpose of this pack

This pack converts the supplied PS 26263 Implementation & Tech Stack Report into a staged, MERN-aligned prototype specification. It is intended to be attached to an AI coding agent as project context. It does not assert that the software already exists.

Evidence labels used in the documents

Label	Meaning	How the agent must treat it
REPORT REQUIREMENT	The project report explicitly describes this capability or workflow.	Preserve its intent, while noting that the original implementation stack may differ.
CURRICULUM ALIGNMENT	The Sigma Batch curriculum includes the relevant learning area.	Prefer these tools where they can safely support the requirement.
PROPOSED IMPLEMENTATION	A practical design choice added to make the MERN prototype buildable.	Implement unless a conflict is identified; do not misrepresent it as an official requirement.
TBD / CONFIRM	A detail cannot be established from the source materials.	Do not guess; ask for the missing authoritative rule or use clearly labelled mock configuration.
NOT IN PROTOTYPE	A production integration or capability intentionally deferred.	Do not fake a successful real integration. Use a named mock adapter if needed.

2. File list and when to attach each PDF

File	Contains	Attach when
01_Problem_Statement_and_Product_Scope.pdf	Problem, intended outcome, user groups, in/out scope and demo flow.	Always; this defines what the product is.
02_Tech_Stack_and_Repository_Spec.pdf	Approved stack, package boundaries, folder layout and dependency rules.	Always; this prevents stack drift.
03_Authentication_and_Authorization_Spec.pdf	Eight roles, session design, permission matrix, auth endpoints and tests.	Before login and any protected page/API.
04_Role_Features_and_Button_Inventory.pdf	Screen-by-screen button/action inventory for every role.	Before frontend pages and corresponding APIs.
05_System_Architecture_and_Workflows.pdf	System diagram, modules, data flow and sequence flows.	Before repository setup and architecture work.
06_Database_and_API_Contract.pdf	MongoDB collections, fields, indexes and proposed REST API contract.	Before database models and backend routes.
07_Validation_Seat_Ledger_and_Conflicts.pdf	Validation layers, severity, seat rules, lifecycle and conflict resolution.	Before upload, admission logic and commit.
08_Security_Privacy_Testing_and_Acceptance.pdf	Security requirements, test cases, go/no-go criteria and limitations.	Throughout; mandatory before demo/deployment.
09_Vibecoding_Prompt_Pack_and_Build_Order.pdf	Copy-ready prompts and milestone acceptance gates.	When asking the agent to implement a new phase.
10_Synthetic_Demo_Data_and_Scenarios.pdf	Safe fake data schema, seed accounts, fixtures and repeatable demo scenarios.	Before creating seed data, validation fixtures or the judge demo.

3. Recommended attachment order

1. Planning only: attach 00, 01, 02 and 05. Ask the agent to produce a plan and list unresolved decisions; do not permit code generation yet.

2. Foundation: attach 01, 02, 03 and 05. Generate the skeleton, environment setup, database connection and authentication only.

3. Each feature: attach its relevant specification plus 04 (button behaviours), 06 (API/data contract), 07 (admission logic) and 08 (acceptance tests) as applicable.

4. Before review: attach 08 and ask for a defect-only audit. Do not mix a code audit with adding new features.

5. Before demo: attach 01, 04, 07, 08 and 10 to verify end-to-end scenarios and disclose all mock services.

4. Non-negotiable project invariants

- Stack: React + JavaScript + Bootstrap; Node.js + Express; MongoDB + Mongoose. No TypeScript, NestJS, PostgreSQL, Redis/BullMQ, Keycloak, Python/ML or Kubernetes unless explicitly approved as a later phase.
- Use synthetic student data in the prototype. Do not place actual APAAR, Aadhaar, DOB or other sensitive identifiers into the demo.
- Real APAAR, AICTE and DigiLocker access is not assumed. External systems must be behind adapters; demo adapters must be named and labelled mock.
- Seat capacities and policy rules must come from a clearly identified configuration/approval record. No unverified legal percentages may be hard-coded.
- Only the server authoritatively validates inputs, checks permissions, reserves seats and commits records.
- Risk scores are investigation-priority signals, not a probability of fraud and not a disciplinary decision.
- A normal QR URL is not a signed verifiable credential. Clearly label prototype QR behaviour.
- Do not report tests, integrations or deployment as successful unless actually executed and verified.

5. Definition of a finished milestone

- Acceptance criteria are written and met.
- Relevant tests were executed and their results are truthful.
- The app starts using documented commands.
- No secrets or live student information are committed.
- The agent lists changed files, new dependencies, assumptions, errors and known limitations.
- A Git checkpoint is created before moving to the next risky phase.

6. Source materials

Primary product source: PS 26263 - Implementation & Tech Stack Report, 33 pages. The source describes the verified student register, eight user types, upload/validation/certification pipeline, category-based seat ledger, conflict resolution, dashboards, consent, QR credentials, audit chain, prototype limitations and proposed production design.

Learning-scope source: Sigma12 Curriculum.pdf, 14 pages. Relevant modules include HTML/CSS, Bootstrap, JavaScript, async/await, AJAX, Git, Node.js, Express, EJS, MongoDB, Mongoose, APIs, error handling, authentication and React. The requested implementation uses React with JavaScript and Bootstrap for the frontend.